

COMPARABLE INTERFACE

Comparable interface:

- It is present in java.lang package.
- The fully qualified name is java.lang.comparable

Member of Comparable Interface:

It has one abstract method which is, **public int compareTo(Object o);**

Why to implement Comparable interface?

- ❖ To compare the attributes of the object, we over-ride equals(Object o) of Object class.
- ❖ But, to compare and sort the elements, we should inherit and over-ride compareTo(Object o) of Comparable interface.
- ❖ **An object can compare itself with another object of same type.**

Design tips to over-ride compareTo(Object o) :

1. The class should be made as child class to Comparable interface,
class ClassName implements Comparable
2. compareTo(Object o) should compare the properties of the current object (self) with the passed object using down-casted reference.
3. The method should return an integer as follows:
 - ❖ 0 if both states are same
 - ❖ +ve integer if current object state is higher
 - ❖ -ve integer if current object state is lower

```
ObjectClassD... ObjectClassD... TreeSetEg.java SortObjects... Implemen
1 package collectionInterface;
2
3 import java.util.Arrays;
4
5 class Book implements Comparable
6 {
7     String name;
8     int pages;
9     Book(){ }
10    Book(String name, int pages){
11        this.name = name;
12        this.pages= pages;
13    }
14    public String toString() {
15        return "name: " + name + " pages: " + pages;
16    }
17    public int compareTo(Object o) {
18        Book dc = (Book)o;
19        // if(this.pages>dc.pages)
20        //     return 1;
21        // if(this.pages<dc.pages)
22        //     return -1;
23        // else
24        //     return 0;
25        // return this.pages==dc.pages;
26        return this.name.compareTo(dc.name);
27    }
28 }
29
```

```

30 class ImplementComparable {
31     public static void main(String[] args) {
32         Book b[] = new Book[4];
33         b[0] = new Book("troy", 120);
34         b[1] = new Book("tata", 82);
35         b[2] = new Book("bata", 45);
36         b[3] = new Book("life", 180);
37         for(int i=0; i<b.length; i++) {
38             System.out.println(b[i]);
39         }
40         System.out.println(Arrays.toString(b));
41         System.out.println(Arrays.asList(b));
42         //Arrays.sort(b);
43         // CLASS CAST EXCEPTION // RUN TIME EXCEPTION
44         Arrays.sort(b);
45         for(int i=0; i<b.length; i++) {
46             System.out.println(b[i]);
47         }
48         Book b1 = new Book();
49         Book b2 = new Book();
50         b1.name = "java";
51         b2.name = "javascript";
52         System.out.println(b1.compareTo(b2));
53     }
54 }
55

```

<terminated> ImplementComparable [Java Application] C:\Users\naray\.p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64_21.0.6.v20.

```

name: troy pages: 120
name: tata pages: 82
name: bata pages: 45
name: life pages: 180
[name: troy pages: 120, name: tata pages: 82, name: bata pages: 45, name: life pages: 180]
[name: troy pages: 120, name: tata pages: 82, name: bata pages: 45, name: life pages: 180]
name: bata pages: 45
name: life pages: 180
name: tata pages: 82
name: troy pages: 120

```

COMPARATOR INTERFACE

Comparator Interface:

- It is present in java.util package.
- The fully qualified name is java.util.Comparator

Members of Comparator Interface:

It has two abstract methods:

- public boolean equals(Object o);
- **public int compare(Object o1, Object o2);**

Why to implement Comparator interface?

- ❖ To compare two objects using a third object, compare(Object o1, Object o2) of Comparator interface should be inherited and over-ridden.
- ❖ Even if two objects to be compared do not have Comparable instance, that is if they are not of Comparable type, they can be compared and sorted.

Design tips to over-ride compare(Object o1, Object o2) :

1. A new implementing class for each attribute can be created to implement Comparator.
2. compare(Object o1, Object o2) should compare the properties of first object with the second object using two down-casted references.
3. The method should return an integer as follows:
 - ❖ 0 if both states are same
 - ❖ +ve integer if current object state is higher
 - ❖ -ve integer if current object state is lower

```

1 package collectionInterface;
2
3 import java.util.Arrays;
4
5
6 class Products{
7     int pid;
8     String name;
9     Products(int pid, String name){
10         this.pid = pid;
11         this.name = name;
12     }
13     public String toString() {
14         return "ID:" + pid + " name:" + name;
15     }
16 }
17 class IdSort implements Comparator{
18     public int compare(Object o1, Object o2) {
19         Products p1 = (Products)o1;
20         Products p2 = (Products)o2;
21         return p1.pid - p2.pid;
22     }
23 }
24 class NameSort implements Comparator{
25     public int compare(Object o1, Object o2) {
26         Products p1 = (Products)o1;
27         Products p2 = (Products)o2;
28         return p1.name.compareTo(p2.name);
29     }
30 }
31 class ImplementComparator {
32     public static void main(String[] args) {
33         Products p[] = new Products[4];
34         p[0] = new Products(101, "headphones");
35         p[1] = new Products(104, "television");
36         p[2] = new Products(103, "laptop");
37         p[3] = new Products(102, "camera");
38         //System.out.println(Arrays.toString(p));
39         System.out.println("*****WITHOUT SORT*****");
40         for(int i=0; i<p.length; i++) {
41             System.out.println(p[i]);
42         }
43         System.out.println("*****ID SORT*****");
44         Arrays.sort(p, new IdSort());
45         for(int i=0; i<p.length; i++) {
46             System.out.println(p[i]);
47         }
48         System.out.println("*****NAME SORT*****");
49         Arrays.sort(p, new NameSort());
50         for(int i=0; i<p.length; i++) {
51             System.out.println(p[i]);
52         }
53     }
54 }
55
56
57

```

<terminated> ImplementComparator [Java

*****WITHOUT SORT*****

ID:101 name:headphones

ID:104 name:television

ID:103 name:laptop

ID:102 name:camera

*****ID SORT*****

ID:101 name:headphones

ID:102 name:camera

ID:103 name:laptop

ID:104 name:television

*****NAME SORT*****

ID:102 name:camera

ID:101 name:headphones

ID:103 name:laptop

ID:104 name:television

COMPARABLE COMPARATOR USING PRIVATE (HIDDEN) ATTRIBUTES:

```
1 package collectionInterface;
2
3 import java.util.Arrays;
4 import java.util.Comparator;
5
6 class StudentC
7 {
8     private int sid;
9     private String name;
10 StudentC(int sid, String name){
11     this.sid = sid;
12     this.name = name;
13 }
14 public int getSid() {
15     return sid;
16 }
17 public void setSid(int sid) {
18     this.sid = sid;
19 }
20 public String getName() {
21     return name;
22 }
23 public void setName(String name) {
24     this.name = name;
25 }
26 public String toString() {
27     return "sid: " + sid + " name: " + name;
28 }
29 }
```

```
30
31 class IdPrivateSort implements Comparator<StudentC>
32 {
33     public int compare(StudentC s1, StudentC s2) {
34         return s1.getSid() - s2.getSid();
35     }
36 }
37
38 class NamePrivateSort implements Comparator<StudentC>
39 {
40     public int compare(StudentC s1, StudentC s2) {
41         return s1.getName().compareTo(s2.getName());
42     }
43 }
44 class ImplementComparatorPrivate {
45     public static void main(String[] args) {
46         StudentC s[] = new StudentC[4];
47         s[0] = new StudentC(101, "Tom");
48         s[1] = new StudentC(104, "Jerry");
49         s[2] = new StudentC(103, "Popeye");
50         s[3] = new StudentC(102, "Bluto");
51         //System.out.println(Arrays.toString(s));
52         System.out.println("*****WITHOUT SORT*****");
53         for(int i=0; i<s.length; i++) {
54             System.out.println(s[i]);
55         }
56         System.out.println("*****ID SORT*****");
57         Arrays.sort(s, new IdPrivateSort());
58         for(int i=0; i<s.length; i++) {
59             System.out.println(s[i]);
60         }
61         System.out.println("*****NAME SORT*****");
62         Arrays.sort(s, new NamePrivateSort());
63         for(int i=0; i<s.length; i++) {
64             System.out.println(s[i]);
65         }
66     }
67 }
```

<terminated> ImplementComparatorPrivate [Java Applica

*****WITHOUT SORT*****

sid: 101 name: Tom
sid: 104 name: Jerry
sid: 103 name: Popeye
sid: 102 name: Bluto

*****ID SORT*****

sid: 101 name: Tom
sid: 102 name: Bluto
sid: 103 name: Popeye
sid: 104 name: Jerry

*****NAME SORT*****

sid: 102 name: Bluto
sid: 104 name: Jerry
sid: 103 name: Popeye
sid: 101 name: Tom